LEMBAR KERJA PRAKTIKUM

Pemrograman Mobile

Pertemuan 10 Integrasi Firebase (Auth dan Firestore)

Mata Kuliah	Pemrograman Mobile
Pertemuan	Pertemuan 10
Topik	Integrasi Firebase (Auth dan Firestore)
Studi Kasus Project	TokoKita, login pengguna dan sinkronisasi pesanan ke cloud
Framework	Flutter (Dart) dan Firebase
Alokasi Waktu	170 menit (3 SKS)
Nama Mahasiswa	Dimas Panca Pamungkas
NIM	3337240063

A. Tujuan Praktikum

- Mahasiswa mampu melakukan setup project Firebase dan menghubungkannya ke proyek Flutter.
- Mahasiswa mampu menerapkan autentikasi pengguna (registrasi dan login) menggunakan Firebase Authentication berbasis email dan password.
- Mahasiswa mampu melakukan operasi CRUD dasar pada Cloud Firestore.
- Mahasiswa mampu merancang struktur data koleksi dan dokumen pada Firestore sesuai kebutuhan aplikasi TokoKita.

B. Alat dan Bahan

- Laptop atau PC dengan proyek tokokita beserta fitur checkout dan riwayat pesanan lokal hasil Pertemuan 9
- Visual Studio Code dan Flutter SDK
- Node.js dan Firebase CLI untuk menjalankan Firebase Emulator (Auth dan Firestore) beserta JDK 21
- Chrome dengan tampilan mobile sebagai pengganti emulator Android
- Package firebase_core, firebase_auth, dan cloud_firestore pada pubspec.yaml

C. Dasar Teori Singkat

Riwayat pesanan TokoKita pada Pertemuan 9 disimpan di SQLite pada satu perangkat, sehingga tidak bisa dibuka dari perangkat lain dan hilang jika aplikasi dihapus. Firebase menyediakan Authentication untuk akun email dan password serta Cloud Firestore sebagai database dokumen. Setiap pesanan disimpan sebagai dokumen pada koleksi orders dengan field uid pemiliknya, lalu StreamBuilder membaca perubahannya secara real-time. Pada praktikum ini koneksi diarahkan ke Firebase Emulator sehingga tidak memerlukan akun Google, dan kodenya tetap sama jika nanti dipindah ke project Firebase asli.

D. Langkah Kerja

Langkah 1. Setup Firebase

- Menambahkan firebase_core, firebase_auth, dan cloud_firestore lalu menjalankan flutter pub get. Package sqflite dihapus karena riwayat pesanan pindah ke cloud.
- Saya tidak menjalankan flutterfire configure karena perintah itu membutuhkan login akun Google dan project di Firebase Console. Sebagai gantinya firebase_options.dart saya tulis manual untuk project demo bernama demo-tokokita, dan koneksi diarahkan ke Firebase Emulator (Auth port 9099 dan Firestore port 8080).
- Memanggil initFirebase pada main sebelum runApp. Log menampilkan Firebase siap dengan project demo-tokokita tanpa error.
- Untuk memakai project asli cukup menjalankan flutterfire configure lalu mengubah kPakaiEmulator menjadi false pada firebase_setup.dart.

Langkah 2. Registrasi dan Login

- Membuat AuthController (GetX) berisi daftar, masuk, dan keluar yang memanggil createUserWithEmailAndPassword, signInWithEmailAndPassword, dan signOut.
- Membuat LoginPage dan RegisterPage. Halaman awal ditentukan di main.dart dengan mengecek currentUser, jika belum login membuka LoginPage dan jika sudah membuka MainPage.
- Memetakan kode FirebaseAuthException ke pesan berbahasa Indonesia, misalnya email-already-in-use dan wrong-password.

Langkah 3. Sesi Login pada MainPage

- Setelah login atau daftar berhasil, pengguna diarahkan ke MainPage dengan Get.offAll.
- Menambahkan tombol Keluar pada Profil yang memanggil signOut, mengosongkan keranjang, lalu Get.offAll ke LoginPage.
- Menampilkan email pengguna pada Profil lewat Obx yang membaca user di AuthController.

Langkah 4. Pesanan di Cloud Firestore

- Merancang koleksi orders dengan field uid, tanggal, daftarItem, totalHarga, dan status.
- Membuat OrderService sebagai pengganti OrderDatabase SQLite. Checkout memanggil simpanDariKeranjang dengan uid pengguna yang login, lalu keranjang dikosongkan.
- Mengubah OrderHistoryPage memakai StreamBuilder dengan query where uid sama dengan uid pengguna. Pengurutan terbaru di atas saya lakukan di aplikasi supaya tidak membutuhkan composite index.
- Tombol Tandai selesai memanggil update status dan tombol hapus memanggil delete pada dokumen.
- Menulis firestore.rules supaya create, read, update, dan delete hanya boleh dilakukan pemilik uid.

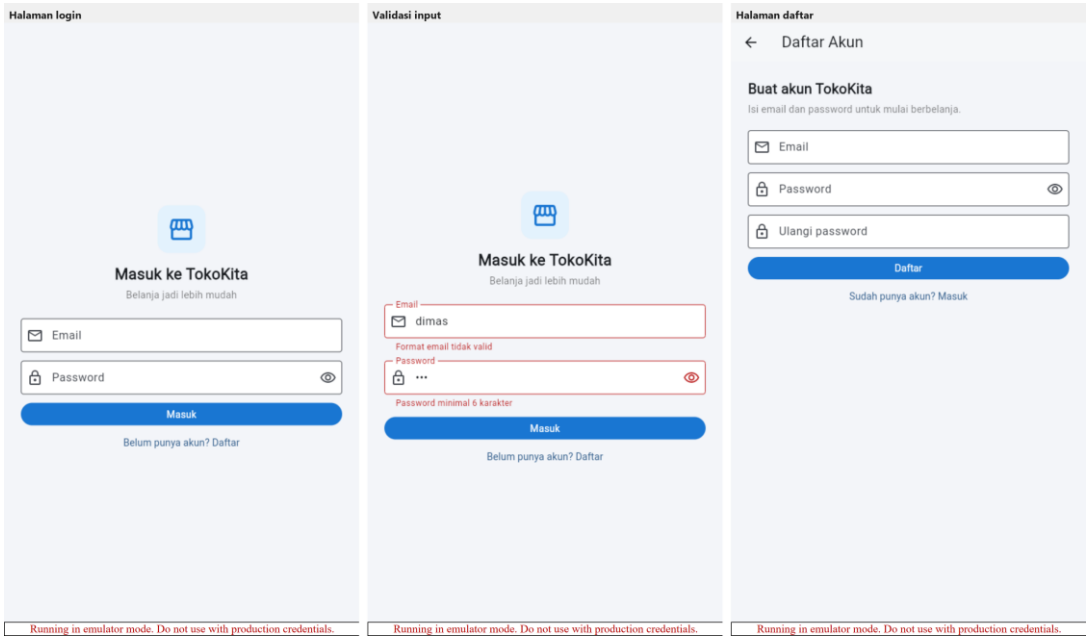
E. Hasil Pengamatan

Pengujian dilakukan pada Firebase Emulator, bukan pada project Firebase asli. Banner Running in emulator mode di bagian bawah tangkapan layar berasal dari Firebase Auth Emulator.

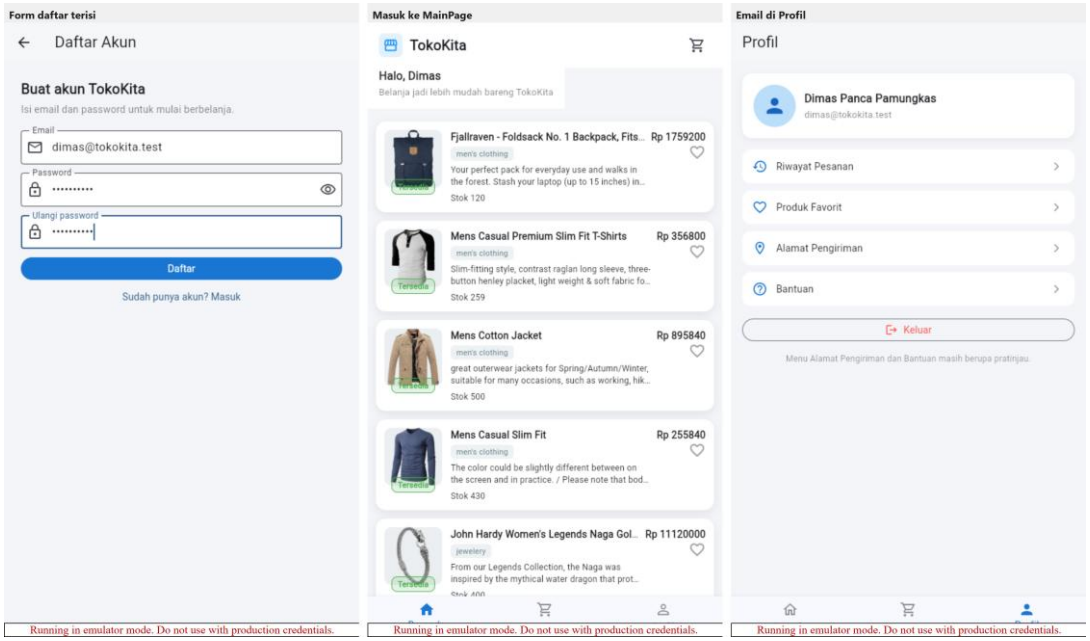
No.	Fitur yang Diuji	Hasil atau Perilaku yang Diamati	Screenshot
1	Konfigurasi FlutterFire dan inisialisasi Firebase	Firebase berhasil diinisialisasi tanpa error dan log mencatat koneksi ke emulator. flutterfire configure tidak dijalankan karena tidak memakai project asli.	Ya
2	Registrasi dan login akun TokoKita	Daftar akun dimas@tokokita.test berhasil dan langsung masuk ke MainPage. Password salah menampilkan Email atau password salah, dan mendaftar dengan email yang sama menampilkan Email sudah terdaftar. Login ulang berhasil.	Ya
3	Logout dan pengalihan ke halaman login	Tombol Keluar pada Profil mengembalikan pengguna ke halaman login dan keranjang menjadi kosong.	Ya
4	Checkout tersimpan ke Firestore (per pengguna)	Checkout 2 item Fjallraven dengan total Rp 3518400 membuat satu dokumen baru pada orders dengan uid akun pertama. Keranjang kosong setelahnya.	Ya
5	Riwayat pesanan real-time via StreamBuilder	Pesanan tampil dari stream. Tombol Tandai selesai mengubah status dari Diproses menjadi Selesai di layar tanpa memuat ulang halaman. Akun kedua melihat 0 pesanan.	Ya

Bukti tampilan aplikasi (mobile view)

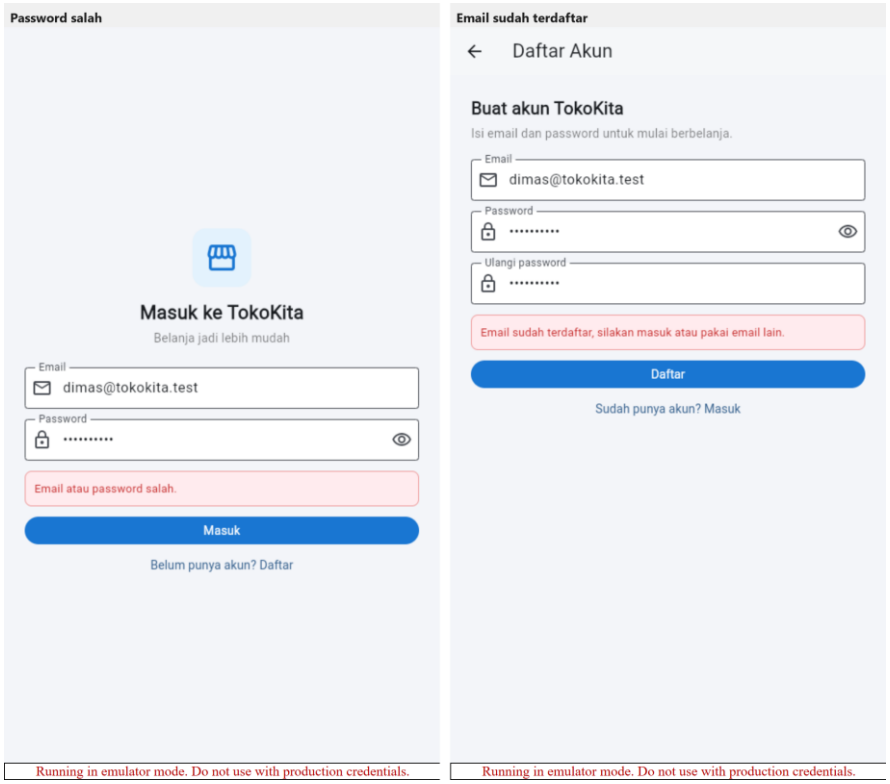
1. Halaman login, validasi input, dan halaman daftar



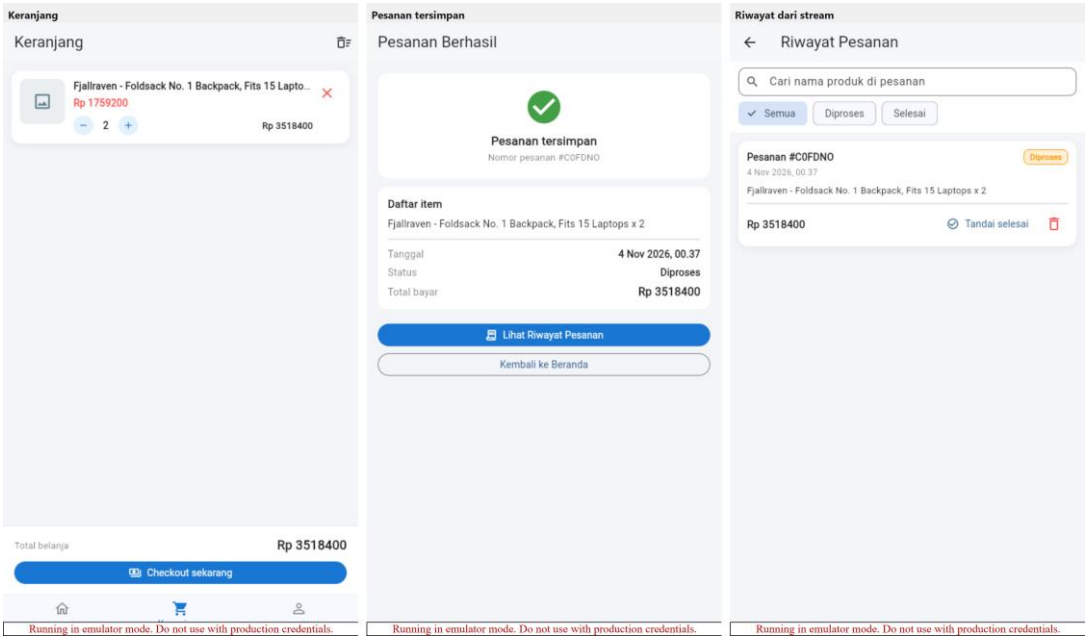
2. Daftar berhasil, masuk ke MainPage, dan email di Profil



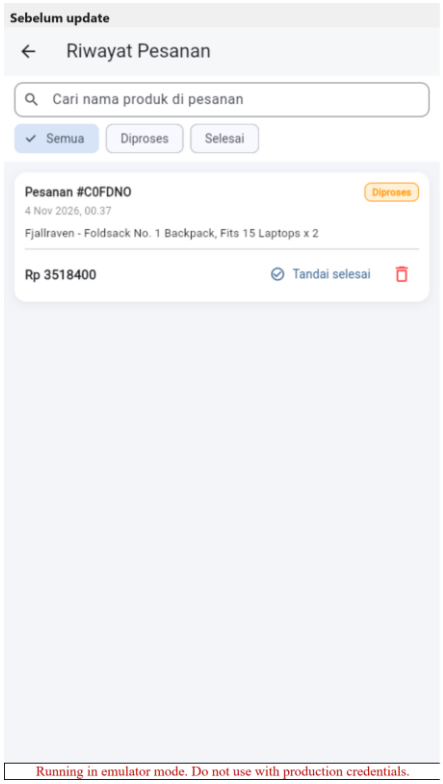
3. Pesan error password salah dan email sudah terdaftar

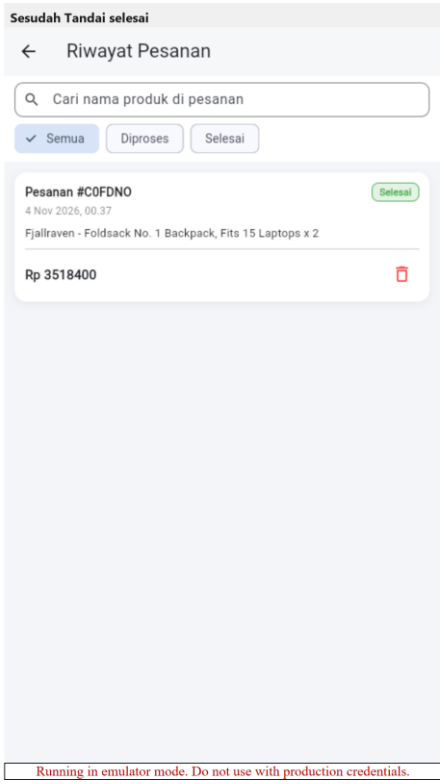


4. Keranjang, pesanan tersimpan, dan riwayat dari stream

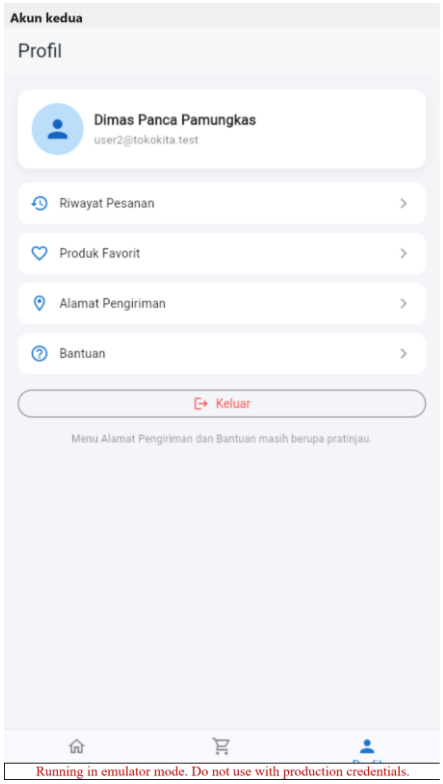


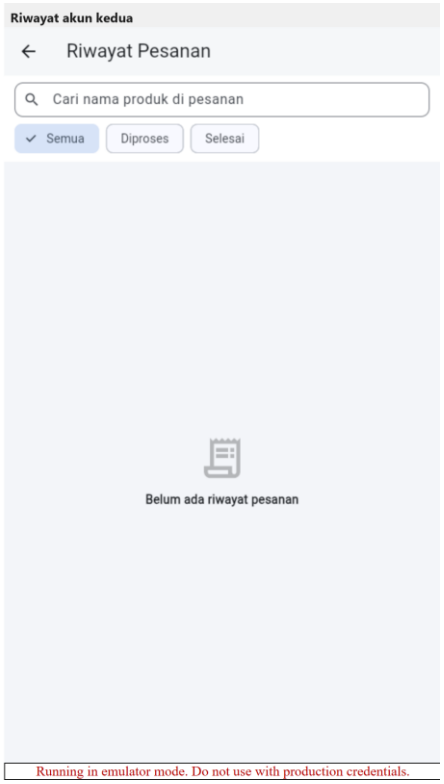
5. Status pesanan berubah menjadi Selesai





6. Akun kedua dengan riwayat pesanan kosong





7. Registrasi, login, dan logout pada AuthController

```
// controllers/auth_controller.dart
await _auth.createUserWithEmailAndPassword(email: e, password: p);
await _auth.signInWithEmailAndPassword(email: e, password: p);
await _auth.signOut();

// pesan error untuk pengguna
case 'email-already-in-use':
  return 'Email sudah terdaftar, silakan masuk atau pakai email lain.';
case 'invalid-credential':
case 'wrong-password':
  return 'Email atau password salah.';
```

8. Operasi Firestore pada OrderService

```
// services/order_service.dart
final _koleksi = FirebaseFirestore.instance.collection('orders');

simpan -> _koleksi.add({uid, tanggal, daftarItem, totalHarga, status})
stream -> _koleksi.where('uid', isEqualTo: uid).snapshots()
update -> _koleksi.doc(id).update({'status': statusBaru})
hapus  -> _koleksi.doc(id).delete()
```

9. Aturan keamanan Firestore

```
// firestore.rules
match /orders/{orderId} {
  allow create: if request.auth != null
    && request.resource.data.uid == request.auth.uid;
  allow read, update, delete: if request.auth != null
    && resource.data.uid == request.auth.uid;
}
```

10. Log terminal

```
# log terminal flutter run (uid dipotong)
Firebase terhubung ke emulator di localhost
Firebase siap, project demo-tokokita
Auth registrasi berhasil dimas@tokokita.test
Auth logout berhasil
Auth gagal wrong-password
Auth login berhasil dimas@tokokita.test
Firestore tambah order g1UhhW6M... total=3518400
Firestore stream orders uid=ibDOpSFa... -> 1 dokumen
Firestore update order g1UhhW6M... status=Selesai
Auth gagal email-already-in-use
Auth registrasi berhasil user2@tokokita.test
Firestore stream orders uid=cPF1VSpt... -> 0 dokumen
```

11. Uji aturan Firestore lewat REST

```
# uji aturan Firestore lewat REST (emulator)
akun1 baca pesanan sendiri   -> 200
akun2 baca pesanan akun1     -> 403 PERMISSION_DENIED
akun2 query uid milik akun1  -> 403
akun2 ubah status pesanan 1  -> 403 PERMISSION_DENIED
tanpa login baca pesanan     -> 403 PERMISSION_DENIED
```

Cara menjalankan aplikasi

Jalankan Firebase Emulator untuk Auth dan Firestore memakai firebase.json yang ada di folder tokokita (membutuhkan JDK 21), lalu jalankan flutter run -d chrome dengan tampilan mobile. Untuk project Firebase asli jalankan flutterfire configure dan ubah kPakaiEmulator menjadi false.

F. Tugas Mandiri

1. Pastikan alur registrasi, login, checkout, lihat riwayat pesanan, dan logout pada TokoKita berjalan lengkap tanpa error.

Jawaban. Alur saya jalankan berurutan mulai dari daftar akun, masuk ke MainPage, tambah produk ke keranjang, checkout, buka riwayat, keluar, lalu masuk lagi. Tidak ada error pada console dan log mencatat setiap langkahnya. Buktinya ada pada bukti tampilan nomor 2, 4, dan 10. Semua ini diuji pada emulator, belum pada project Firebase asli.

2. Tambahkan validasi input sederhana pada form registrasi/login (misalnya format email dan panjang minimal password) sebelum data dikirim ke Firebase.

Jawaban. Validasi dijalankan pada form login dan daftar sebelum memanggil Firebase. Email harus berformat nama@domain.tld lewat regex dan password minimal 6 karakter. Form daftar punya tambahan ulangi password yang harus sama. Pesan yang muncul antara lain Format email tidak valid dan Password minimal 6 karakter (bukti nomor 1). Fungsinya saya pisahkan menjadi validasiEmail dan validasiPassword supaya bisa diuji, dan 6 unit test untuk keduanya lulus.

3. Pastikan setiap pengguna hanya dapat melihat riwayat pesanan miliknya sendiri, lalu uji dengan mendaftarkan minimal dua akun berbeda.

Jawaban. Saya mendaftarkan dua akun, yaitu dimas@tokokita.test dan user2@tokokita.test. Akun pertama punya 1 pesanan sedangkan akun kedua melihat riwayat kosong (bukti nomor 6). Log stream juga mencatat 1 dokumen untuk akun pertama dan 0 dokumen untuk akun kedua. Filternya memakai where uid pada query, dan sebagai lapisan kedua firestore.rules menolak akses lintas akun. Lewat REST, akun kedua yang mencoba membaca atau mengubah pesanan akun pertama mendapat 403 PERMISSION_DENIED (bukti nomor 11).

4. Jelaskan struktur koleksi orders yang Anda rancang pada Firestore, beserta alasan pemilihan field dan cara mengaitkannya dengan pengguna yang login.

Jawaban. Koleksi orders berisi satu dokumen per pesanan dengan ID otomatis dari Firestore. Field uid menyimpan ID pengguna dari Firebase Auth sebagai penghubung ke pemilik pesanan. Field tanggal disimpan sebagai Timestamp supaya bisa diurutkan, daftarItem berupa teks ringkasan item, totalHarga berupa angka, dan status bernilai Diproses atau Selesai. Saya memilih satu koleksi datar dengan field uid dibanding subkoleksi per pengguna karena query dan aturan keamanannya lebih sederhana. Menurut saya itu sudah cukup untuk skala TokoKita.

5. Menurut pemahaman Anda, apa perbedaan mendasar antara Cloud Firestore dan basis data SQLite yang telah dipelajari pada Pertemuan 9, khususnya terkait sinkronisasi antar perangkat dan kebutuhan akun pengguna?

Jawaban. Menurut saya perbedaan utamanya ada pada lokasi data. SQLite menyimpan data di satu perangkat sehingga data tidak ikut berpindah jika aplikasi dihapus atau pengguna ganti HP. Firestore menyimpan data di server sehingga bisa dibaca dari perangkat mana pun dan perubahannya langsung muncul lewat stream. Karena datanya ada di server yang dipakai bersama, Firestore membutuhkan akun pengguna dan aturan keamanan supaya tiap orang hanya melihat datanya sendiri. SQLite tidak membutuhkan itu karena datanya sudah terpisah di tiap perangkat. Sinkronisasi antar dua perangkat belum saya uji karena saya hanya memakai satu browser, jadi bagian itu berdasarkan konsep.

G. Format Pengumpulan

- Laporan dikumpulkan dalam format PDF.
- Format nama file Praktikum10_NIM_NamaMahasiswa.pdf
- Dikumpulkan paling lambat sebelum pertemuan berikutnya dimulai.